ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Design and Analysis of Algorithms

Lecture 2 and Lecture 3: Asymptotic Analysis (2)

© Fall 2025 – **Dr. Dalia Ezzat**
dalia.ezzat@sut.edu.eg

Excerpts from the previous lecture

- We primarily analyze Time complexity (runtime) over Space complexity.
- We use Theoretical (Asymptotic) Analysis because the Naïve approach is unreliable (dependent on hardware, implementation, and specific input data).
- Time Complexity is calculated by counting the number of primitive operations (such as addition, indexing, comparison) as a function of the input size.

Excerpts from the previous lecture

- **Time Complexity Estimation Rules:**

  - ✓ Consecutive statements: Add the running times.
  - ✓ Loops/Nested Loops: Time is proportional to the number of iterations.
  - ✓ If/Else: Use the running time of the larger of the two branches (worst-case path).

- **The most important factor is the growth rate.**

Excerpts from the previous lecture

- **Growth Rate Simplification:**
  - ✓ Ignore Low-Order Terms: Only use the higher-order term.
  - ✓ Ignore Constants: Ignore multiplicative constants.

Excerpts from the previous lecture

- **Worst-Case Analysis:** The maximum amount of time that an algorithm require to solve a problem of size n.
  - ✓ This gives an upper bound for the time complexity of an algorithm.
  - ✓ Normally, we try to find worst-case behavior of an algorithm.
- **Best-Case Analysis:** The minimum amount of time that an algorithm require to solve a problem of size n.
  - ✓ The best-case behavior of an algorithm is NOT so useful.
- **Average-Case Analysis** : The average amount of time that an algorithm require to solve a problem of size n.
  - ✓ Sometimes, it is difficult to find the average-case behavior of an algorithm.
  - ✓ We have to look at all possible data organizations of a given size n, and their distribution probabilities of these organizations.
- **Worst-case analysis is more common than others.**

Table of Contents

Recursion and Comparison with Loops

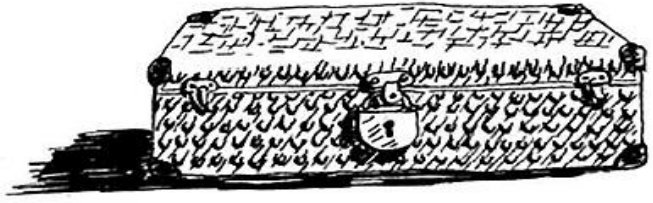
- Recursion is an elegant way to solve problems and is a valuable coding technique used in many algorithms.
- **Both recursion and iteration (using loops) can be used to accomplish the same tasks.**
- There is typically no inherent performance advantage to using recursion; sometimes, loops are actually better for performance.
- **“Loops may achieve a performance gain for your program. Recursion may achieve a performance gain for your programmer”.** You must choose which priority is more important in your specific situation.

Recursion and Comparison with Loops

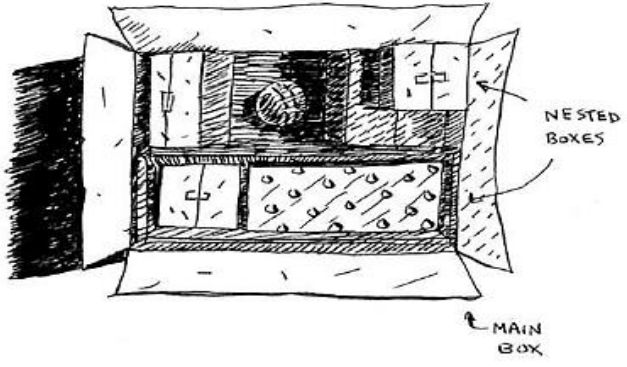
```
def look_for_key(main_box):
  pile = main_box.make_a_pile_to_look_through()
  while pile is not empty:
    box = pile.grab_a_box()
    for item in box:
      if item.is_a_box():
        pile.append(item)
      elif item.is_a_key():
        print "found the key!"
```

```
def look_for_key(box):
  for item in box:
    if item.is_a_box():
      look_for_key(item)   <-- Recursion!
    elif item.is_a_key():
      print "found the key!"
```

Suppose you're digging through your grandma's attic and come across a mysterious locked suitcase.



Grandma tells you that the key for the suitcase is probably in this other box.



This box contains more boxes, with more boxes inside those boxes. The key is in a box somewhere. What's your algorithm to search for the key?

Recursion

- The relationship between recursion and the **call stack** is a core concept.

- The call stack is a fundamental stack data structure that your computer uses internally. **A stack is a simple structure that operates on a LIFO (Last In, First Out) principle**. **It has two main actions: push (insert an item) and pop (remove and read the top item).**

- **All function calls go onto the call stack.** Every time a function is called, the computer allocates a "box" of memory on the stack. This box saves the values of all the variables specific to that function call.

Recursion

- When a function calls another function (or calls itself recursively), the calling function is paused in a partially completed state. Its variable values remain stored in memory until the called function returns.

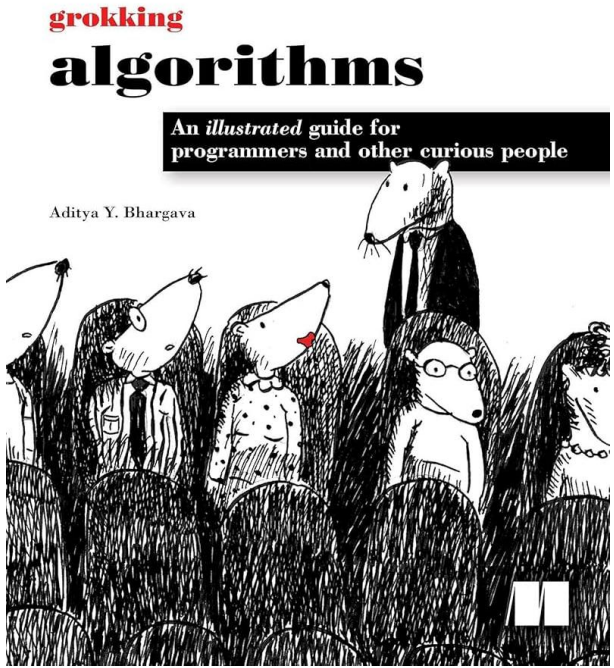
- Recursive functions use the call stack to manage their operations. **In a recursive function, the call stack is vital because it tracks the state of all the recursive calls.**

- The call stack consumes memory. **If a recursive function is written incorrectly and runs forever, the stack will grow indefinitely. If the program runs out of space on the call stack, it will exit with a stack-overflow error.**

Recursion

- For a full and detailed explanation of how the call stack functions with recursion, including step-by-step examples of the factorial function and avoiding infinite loops.

- Please refer to Chapter 3, specifically "The call stack" and "The call stack with recursion" sections, in the Grokking Algorithms book. Pages (42-50).



Recursive Functions

- General form for recursive functions:

$$T(n) = a \cdot T(\textit{smaller size}) + b$$

Where:

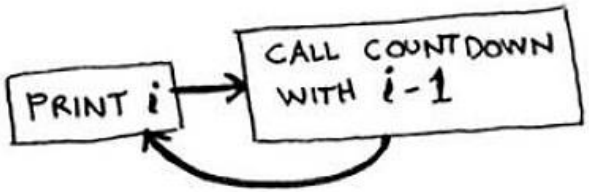
- a: number of recursive calls
- b: cost of non-recursive part

Base case and recursive case

- Because a recursive function calls itself, it's easy to write a function incorrectly that ends up in an infinite loop. For example, suppose you want to write a function that prints a countdown, like this: 3...2...1.
- You can write it recursively, like so:

```
def countdown(i):
    print i
countdown(i-1)
```

- Write out this code and run it. You'll notice a problem: this function will run forever!



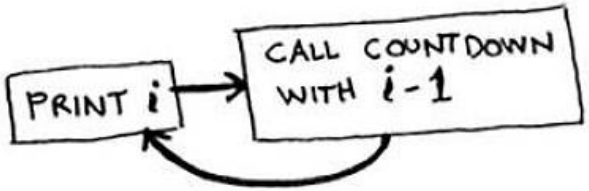
Infinite loop

Base case and recursive case

- When you write a recursive function, you have to tell it when to stop recursing. That's why every recursive function **has two parts: the base case, and the recursive case.**

- The recursive case is when the function calls itself.

- The base case is when the function doesn't call itself again. So it doesn't go into an infinite loop.

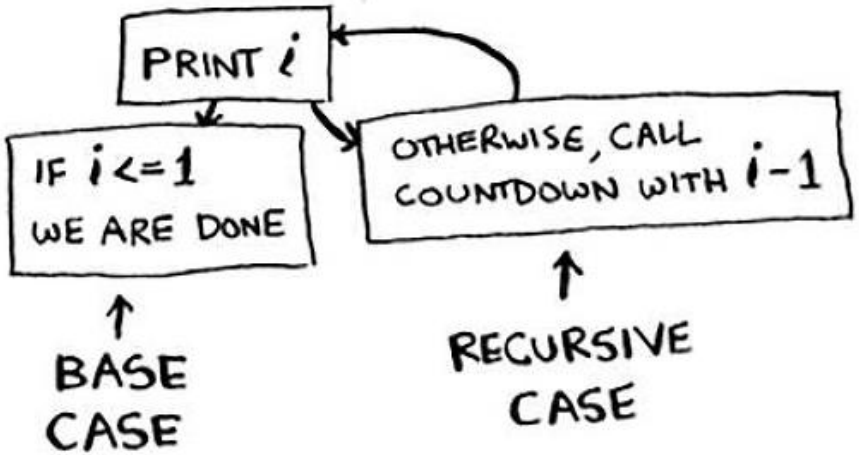


Infinite loop

Base case and recursive case

- **Let's add a base case to the countdown function:**

```
def countdown(i):
  print i
  if i <= 0:          <-- Base case
    return
  else:               <-- Recursive case
    countdown(i-1)
```



- **Now the function works as expected. It goes something like the provided Figure.**

Examples of Recurrence Relations (1/7)

- **Example 1: Basic Recursive Function**

```
void Test(int n) {
    if (n > 0) {
        for(i = 0; i < n; i++) {
            print('id', n);
        }
        Test(n-1);
    }
}
```

Operation	Cost
if (n > 0)	1
for(i=0; i<n; i++)	n+1
print('id', n)	n
Test(n-1)	T(n-1)

Examples of Recurrence Relations (2/7)

- **Example 1: Basic Recursive Function**

We analyze the WORST CASE path (when n > 0):

For n > 0:

$$T(n) = \underbrace{1}_{\text{Line 1}} + \underbrace{(n+1)}_{\text{Line 2}} + \underbrace{n}_{\text{Line 3}} + \underbrace{T(n-1)}_{\text{Line 5}} = T(n-1) + 2n + 2$$

Base Case (n = 0):

$$T(0) = \underbrace{1}_{\text{Line 1}} = 1$$

Complete Recurrence Relation:

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \\ T(n-1) + 2n + 2 & \text{if } n > 0 \end{cases}$$

Operation	Cost
if (n > 0)	1
for(i=0; i<n; i++)	n+1
print('id', n)	n
Test(n-1)	T(n-1)

Examples of Recurrence Relations (3/7)

- **Example 2: Basic Recursive Factorial**

```
int fact(int n) {
    if (n == 0) return 1;
    else return fact(n - 1);
}
```

Operation	Cost	Explanation
if (n == 0)	1	Single comparison
return 1	1	Return constant value
return fact(n - 1)	T(n-1)	Recursive call cost

Recurrence Relation:

$$T(n) = \begin{cases} 2 & \text{if } n = 0 \\ T(n-1) + 1 & \text{if } n > 0 \end{cases}$$

Examples of Recurrence Relations (4/7)

- **Example 2: Modified Factorial**

```
int fact(int n) {
    if (n == 0) return 1;
    else return 4 * fact(n - 1);
}
```

Recurrence Relation:

$$T(n) = \begin{cases} 2 & \text{if } n = 0 \\ T(n-1) + 1 & \text{if } n > 0 \end{cases}$$

Examples of Recurrence Relations (5/7)

- **Example 3: Single Recursive Call**

```
int fact(int n) {
    if (n == 1) return 1;
    else return fact(n/2);
}
```

Recurrence Relation:

$$T(n) = \begin{cases} 2 & \text{if } n = 1 \\ T(n/2) + 1 & \text{if } n > 1 \end{cases}$$

Examples of Recurrence Relations (6/7)

- **Example 4: Double Recursive Calls**

```
int fact(int n) {
    if (n == 1) return 1;
    else return fact(n/2) + fact(n/2);
}
```

Recurrence Relation:

$$T(n) = \begin{cases} 2 & \text{if } n = 1 \\ 2T(n/2) + 1 & \text{if } n > 1 \end{cases}$$

Examples of Recurrence Relations (7/7)

- **Example 5: Fibonacci-like Recursion**

```
int fact(int n) {
    if (n == 1) return 1;
    else return fact(n-1) + fact(n-2);
}
```

The Fibonacci sequence is a type series where each number is the sum of the two that precede it. The Fibonacci sequence is given by 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, and so on.

$$T(n) = T(n-1) + T(n-2) + 1$$

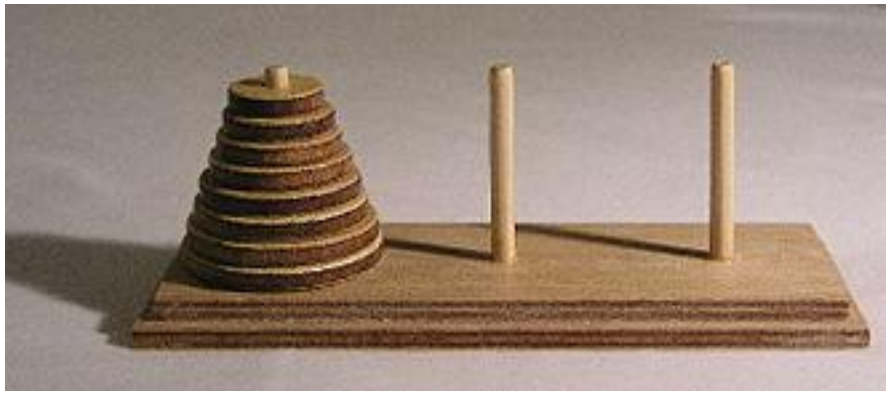
We can write it like this also by considering recursive case only.

Tower of Hanoi Problem (1/7)

- **Tower of Hanoi is a mathematical puzzle with three rods (A, B, C) and N disks. The objective is to move the entire stack from rod A to rod C, following these rules:**

  - Only one disk can be moved at a time.

  - No disk may be placed on top of a smaller disk.



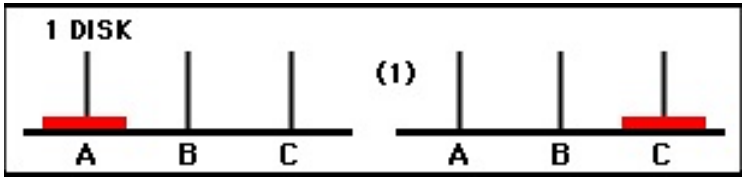
Source A **Spare B** **Destination C**

Tower of Hanoi Problem (2/7)

- Number of disk used is 1. The problem is represented in the figure provided. The disk has to be transferred from position A to C. B's the intermediate rod.

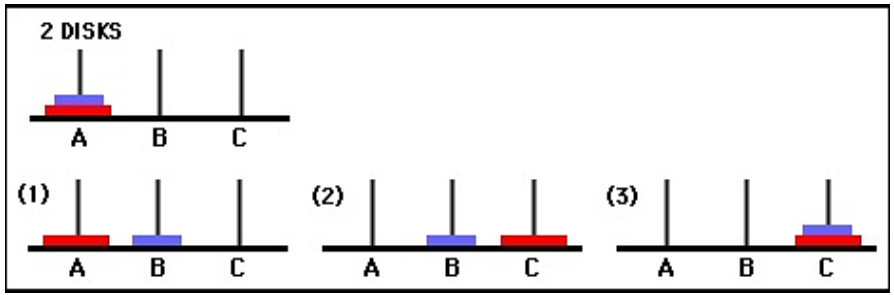
- Here only 1 move is required, and the intermediate rod is not used at all. Red disk is moved directly from A to C.

- Number of move =1.



Tower of Hanoi Problem (3/7)

- Number of disks used are 2. Refer to the image given. Disks are placed at rod A and have to be transferred to C as usual with the largest at the bottom and the smallest at the top.

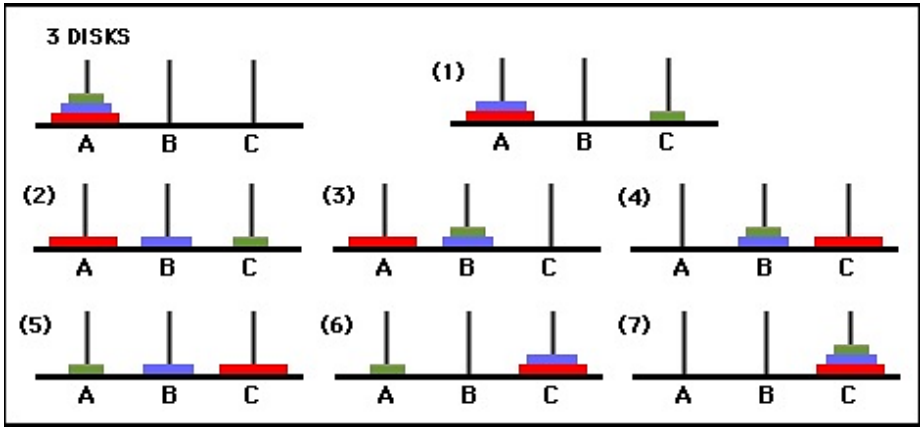


- Blue disk is moved from A to B.
  Red disk is moved from A to C.
  Blue disk is moved from B to C.

- Number of moves= 3.

Tower of Hanoi Problem (4/7)

- Number of disks used are 3. The image is provided below. Disks have to be transferred from A to C with the largest at the bottom and the smallest at the top.



- Green disk is moved from A to C.
  Blue disk is moved from A to B.
  Green disk is moved from C to B.
  Red disk is moved from A to C.
  Green disk is moved from B to A.
  Blue disk is moved from B to C.
  Green disk is moved from A to C.

- Number of moves= 7.

ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Now we'll stop here because the information about this problem is enough to get a formula.

Tower of Hanoi Problem (5/7)

- For 1 disk, number of moves required (M1) =1.
- For 2 disks, number of moves required (M2) =3.

  M2= (2*M1) +1= (2*1) + 1= 3.
- For 3 disks, the number of moves required (M3) = 7.

  M3= (2*M2)+1= (2*3)+1= 7.
- The formula derived should be: Required move= (2*Previous move) +1. Thus, we will be able to calculate the successive required moves if we know the previous moves required. Example: For 4 disks, required moves= 2*7+1 = 15. For 5 disks, required moves= 2*15 +1 = 31.

Tower of Hanoi Problem (6/7)

- We can already find a pattern through the information provided to find out an explicit pattern formula.

$$\boldsymbol{Number\ of\ moves = 2^n - 1}$$

- Apply it and you'll find the exact same information on the right-hand side column on the provided table. When disks = 4, Number of moves= (2^4) - 1= 15.

- If numbers of disks involved were 64. Therefore, number of moves= $2^{64} - 1$ = 18,446,744,073,709,551,615.

Explicit Pattern Formula	
Disks(n)	**Moves**
1	1
2	3
3	7
4	15
5	31

Tower of Hanoi Problem (7/7)

```
pseudocode


MoveTower(n, source, dest, spare) {
    IF n == 0, THEN:
        move disk from source to dest
    ELSE:
        MoveTower(n-1, source, spare, dest)
        move disk from source to dest
        MoveTower(n-1, spare, dest, source)
}
```

Recurrence Relation can be written in different way as following:

$$T(n) = 2T(n-1) + 1$$ ✓

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \\ 2T(n-1) + 1 & \text{if } n > 0 \end{cases}$$ ✓

Solving Recurrence Relations

- **Three main methods:**
  - Substitution Method
  - Master Theorem Method
  - Recursion Tree Method

Substitution Method- Example 1 (1/3)

- **Tower of Hanoi Problem**

Recurrence Relation:

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \\ 2T(n-1) + 1 & \text{if } n > 0 \end{cases}$$

Step 1: For n ≥ 1

$$T(n) = 2T(n-1) + 1$$

Step 2: First Substitution

$$\begin{aligned} T(n) &= 2[2T(n-2) + 1] + 1 \\ &= 4T(n-2) + (2+1) \\ &= 4T(n-2) + 3 \end{aligned}$$

Substitution Method- Example 1 (2/3)

- **Tower of Hanoi Problem**

Step 3: Second Substitution

$$\begin{aligned} T(n) &= 4[2T(n-3)+1]+3 \\ &= 8T(n-3)+(4+2+1) \\ &= 8T(n-3)+7 \end{aligned}$$

Step 4: Third Substitution

$$\begin{aligned} T(n) &= 8[2T(n-4)+1]+7 \\ &= 16T(n-4)+(8+4+2+1) \\ &= 16T(n-4)+15 \end{aligned}$$

Substitution Method- Example 1 (3/3)

- **Tower of Hanoi Problem**

Step 5: Pattern Recognition

After k substitutions $= 2^k T(n-k) + 2^{k-1} + 2^{k-2} + \cdots + 2^2 + 2 + 1$

Assume $k = n$, and use $T(0) = 1$:

$$\begin{aligned} T(n) &= 2^n T(0) + 2^{n-1} + 2^{n-2} + \cdots + 2^2 + 2 + 1 \\ &= 2^n + 2^{n-1} + 2^{n-2} + \cdots + 2^2 + 2 + 1 \end{aligned}$$

This is a geometric progression with powers of 2 increasing from 2^0 to 2^n, so:

Time complexity: $T(n) = O(2^n)$

Geometric Series - Increasing Formula (1/3)

Increasing Geometric Series

A geometric series is **increasing** when the common ratio $r > 1$.

General Form

$$S = a + ar + ar^2 + ar^3 + \cdots + ar^{n-1}$$

Where:

- a = first term
- r = common ratio ($r > 1$)
- n = number of terms

Geometric Series - Increasing Formula (2/3)

Sum Formula

For $r > 1$:

$$S_n = a \cdot \frac{r^n - 1}{r - 1}$$

Alternative Forms

$$S_n = a \cdot \frac{1 - r^n}{1 - r} \quad \text{(mathematically equivalent)}$$

Special Case (a = 1)

When the first term is 1:

$$S_n = 1 + r + r^2 + \cdots + r^{n-1} = \frac{r^n - 1}{r - 1}$$

Geometric Series - Increasing Formula (3/3)

Tower of Hanoi Example

In the Tower of Hanoi recurrence:

- $a = 1$
- $r = 2$
- $n = k$ terms

$$S_k = 1 + 2 + 4 + \cdots + 2^{k-1} = \frac{2^k - 1}{2 - 1} = 2^k - 1$$

Substitution Method- Example 2 (1/3)

- **Binary Search Problem**

  Find the time complexity of binary search using the recurrence:

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \text{with} \quad T(1) = 1$$

Step 1: Initial Recurrence

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Step 2: First Substitution

Substitute $T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + 1$:

$$\begin{aligned} T(n) &= \left[T\left(\frac{n}{4}\right) + 1\right] + 1 \\ &= T\left(\frac{n}{4}\right) + 2 \end{aligned}$$

Substitution Method- Example 2 (2/3)

- **Binary Search Problem**

Step 3: Second Substitution

Substitute $T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + 1$:

$$\begin{aligned} T(n) &= \left[T\left(\frac{n}{8}\right) + 1\right] + 2 \\ &= T\left(\frac{n}{8}\right) + 3 \end{aligned}$$

Step 4: Third Substitution

Substitute $T\left(\frac{n}{8}\right) = T\left(\frac{n}{16}\right) + 1$:

$$\begin{aligned} T(n) &= \left[T\left(\frac{n}{16}\right) + 1\right] + 3 \\ &= T\left(\frac{n}{16}\right) + 4 \end{aligned}$$

Substitution Method- Example 2 (3/3)

- **Binary Search Problem**

We continue until we reach the base case where:

As you divide until you reach to 1.

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$$

Final Substitution

$$\begin{aligned} T(n) &= T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n \\ &= T(1) + \log_2 n \\ &= 1 + \log_2 n \end{aligned}$$

Time complexity: $\boxed{T(n) = O(\log n)}$

The Master Method (1/13)

- The master method applies to recurrences of the form:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

where:

- $a \geq 1$
- $b > 1$
- $f(n)$ is asymptotically positive

The Master Method (2/13)

- **Three fundamental case:**

  Compare $f(n)$ with $n^{\log_b a}$:

  1. $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$.
     - $f(n)$ grows polynomially slower than $n^{\log_b a}$ (by an n^{ε} factor).

     Solution: $T(n) = \Theta(n^{\log_b a})$.

The Master Method (3/13)

- **Three fundamental case:**

Compare $f(n)$ with $n^{\log_b a}$:

2. $f(n) = \Theta(n^{\log_b a} \lg^k n)$ for some constant $k \geq 0$.
   - $f(n)$ and $n^{\log_b a}$ grow at similar rates.

   Solution: $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

The Master Method (4/13)

- **Three fundamental case:**

Compare $f(n)$ with $n^{\log_b a}$:

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$.
   - $f(n)$ grows polynomially faster than $n^{\log_b a}$ (by an n^{ε} factor),

   and $f(n)$ satisfies the ***regularity condition*** that $af(n/b) \leq cf(n)$ for some constant $c < 1$.

   Solution: $T(n) = \Theta(f(n))$.

The Master Method (5/13)

- **The simplified form**

For recurrences of the form:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + n^d \log^k n$$

We can simplify the comparison:

1. If $n^{\log_b a} > n^d$, then $T(n) = O(n^{\log_b a})$
2. If $n^{\log_b a} = n^d$, then $T(n) = O(n^d \log^{k+1} n)$
3. If $n^{\log_b a} < n^d$, then $T(n) = O(n^d \log^k n)$

The Master Method (6/13)

Example 1

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

- $a = 4, b = 2$
- $n^{\log_2 4} = n^2$
- Compare: n^2 vs n
- $n^2 > n$
- **Case 1**: $T(n) = \Theta(n^2)$

The Master Method (7/13)

Example 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

- $a = 4, b = 2$
- $n^{\log_2 4} = n^2$
- Compare: n^2 vs n^2
- $n^2 = n^2$
- **Case 2**: $T(n) = \Theta(n^2 \log_2 n)$

The Master Method (8/13)

Example 3

$$T(n) = 4T\left(\frac{n}{2}\right) + n^3$$

- $a = 4, b = 2$
- $n^{\log_2 4} = n^2$
- Compare: n^2 vs n^3
- $n^2 < n^3$
- **Case 3**: $T(n) = \Theta(n^3)$

The Master Method (9/13)

Example 4

$$T(n) = 2T\left(\frac{n}{8}\right) + \sqrt[3]{n}$$

- $a = 2, b = 8$
- $n^{\log_8 2} = n^{1/3}$
- Compare: $n^{1/3}$ vs $n^{1/3}$
- $n^{1/3} = n^{1/3}$
- **Case 2**: $T(n) = \Theta(n^{1/3} \log_8 n)$

The Master Method (10/13)

Example 5

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

- $a = 9, b = 3$
- $n^{\log_3 9} = n^2$
- Compare: n^2 vs n
- $n^2 > n$
- **Case 1**: $T(n) = \Theta(n^2)$

The Master Method (11/13)

Example 6

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

- $a = 1, b = \frac{3}{2}$
- $n^{\log_{3/2} 1} = n^0 = 1$
- Compare: 1 vs 1
- $1 = 1$
- **Case 2**: $T(n) = \Theta(\log_{3/2} n)$

The Master Method (12/13)

Example 7

$$T(n) = 2T\left(\frac{n}{2}\right) + n\log_2 n$$

- $a = 2, b = 2$
- $n^{\log_2 2} = n^1 = n$
- Compare: n vs n
- $n = n$
- **Case 2**: $T(n) = \Theta(n\log_2^2 n)$

The Master Method (13/13)

Example 8

$$T(n) = 2T\left(\frac{n}{4}\right) + n^2 \log_2^3 n$$

- $a = 2, b = 4$
- $n^{\log_4 2} = n^{1/2} = \sqrt{n}$
- Compare: $\sqrt{n}$ vs n^2
- $\sqrt{n} < n^2$
- **Case 3**: $T(n) = \Theta(n^2 \log_2^3 n)$

ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Any Question?